claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-adffdf03c4d89be94.jsonl`
- SHA-256: `2464fc916c4fa86281624d95d4e01f8865e772fccbbd5c1b2662e587d8b9bdb1`
- Source modified: `2026-06-10T06:12:33+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_2a64bc88-2a8`
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`

Transcript

1. user (2026-06-10T06:10:48.443Z)

CONTEXT ? two sibling repos on a Windows machine:
- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).
- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).
Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources.
ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.
QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.
ROLE: adversarial verifier. An auditor ("reliability") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.
FINDING: {"title":"SQLite opened with no WAL and no busy_timeout; concurrent writes raise 'database is locked' and the write helpers swallow it","severity":"high","area":"concurrency / data-integrity","file":"backend/infrastructure/database.py:14-19 (_get_connection), :103-114 (add_video), :116-128 (add_play_segment), :130-141 (add_event)","evidence":"_get_connection does sqlite3.connect + PRAGMA foreign_keys=ON only ? no `journal_mode=WAL`, no `busy_timeout`. add_play_segment/add_event/add_video wrap the write in try/except that logs and returns None/False on ANY exception, including 'database is locked'. Two simultaneous /api/process calls (or multiple uvicorn workers) writing segments will contend; the loser's insert returns None and that segment is silently dropped from segments_data with no error surfaced.","why_it_matters":"Under the very concurrency the async-job slice is meant to enable, customer highlight segments vanish with zero error. DEFERRED #16 only flags check_same_thread / connection-per-thread (plan §3 Risks) ? it does NOT mention WAL, busy_timeout, write contention, or the swallow-and-drop behavior. Incomplete deferral.","why":"placeholder"}
Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:10:53.679Z)

1	import sqlite3

```
      import json
      import os
      import logging
      from typing import List, Dict, Any, Optional

      logger = logging.getLogger(__name__)

      class Database:
          def __init__(self, db_path: str):
              self.db_path = db_path
              self._init_db()

          def _get_connection(self):
              conn = sqlite3.connect(self.db_path)
              conn.row_factory = sqlite3.Row
              # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
              conn.execute("PRAGMA foreign_keys = ON")
              return conn

          def _init_db(self):
              """Initializes tables for videos, play_segments, and events."""
              with self._get_connection() as conn:
                  cursor = conn.cursor()

                  # Videos Table
                  cursor.execute("""
                      CREATE TABLE IF NOT EXISTS videos (
                          id TEXT PRIMARY KEY,
                          filepath TEXT NOT NULL,
                          processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                          status TEXT NOT NULL,
                          validation_results TEXT
                      )
                  """)

                  # Play Segments Table (Extensible metadata JSON)
                  cursor.execute("""
                      CREATE TABLE IF NOT EXISTS play_segments (
                          id INTEGER PRIMARY KEY AUTOINCREMENT,
                          video_id TEXT NOT NULL,
                          start_time REAL NOT NULL,
                          end_time REAL NOT NULL,
                          duration REAL NOT NULL,
                          server TEXT,
                          receiver TEXT,
                          metadata TEXT,
                          FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                      )
                  """)

                  # Events Table
                  cursor.execute("""
                      CREATE TABLE IF NOT EXISTS events (
                          id INTEGER PRIMARY KEY AUTOINCREMENT,
                          segment_id INTEGER NOT NULL,
                          timestamp REAL NOT NULL,
                          type TEXT NOT NULL,
                          player TEXT,
                          details TEXT,
                          FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                      )
                  """)

                  # Versioned migrations live here. PRAGMA user_version is the schema
```

```
66                  # contract ? bump it for every change and add an ordered `if version < N`
67                  # block below. Tests assert the expected version, so accidental drift fails
CI.
68                  version = cursor.execute("PRAGMA user_version").fetchone()[0]
69
70                  if version < 1:
71                      # v1: raw candidate detections (pre-confirmation), detector-agnostic.
72                      # Common fields are columns; detector-specific metrics go in `stats`
JSON,
73                      # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
74                      cursor.execute("""
75                          CREATE TABLE IF NOT EXISTS candidate_segments (
76                              id INTEGER PRIMARY KEY AUTOINCREMENT,
77                              video_id TEXT NOT NULL,
78                              detector TEXT NOT NULL,
79                              chunk_index INTEGER,
80                              start_time REAL NOT NULL,
81                              end_time REAL NOT NULL,
82                              duration REAL NOT NULL,
83                              confidence REAL,
84                              stats TEXT,
85                              detection_params TEXT,
86                              confirmed_segment_id INTEGER,
87                              human_label TEXT,
88                              notes TEXT,
89                              created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
90                              FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
91                              FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
92                          )
93                      """)
94                      cursor.execute("""
95                          CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
96                              ON candidate_segments(video_id, detector)
97                      """)
98                      cursor.execute("PRAGMA user_version = 1")
99                  # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2
100
101                 conn.commit()
102
103         def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
104             try:
105                 with self._get_connection() as conn:
106                     conn.cursor().execute(
107                         "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
108                         (video_id, filepath, status, json.dumps(validation_results))
109                     )
110                     conn.commit()
111                 return True
112             except Exception as e:
113                 logger.error(f"Failed to add video: {e}")
114                 return False
115
116         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
117             try:
118                 with self._get_connection() as conn:
119                     cursor = conn.cursor()
120                     cursor.execute(
121                         "INSERT INTO play_segments (video_id, start_time, end_time,
```

```
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
                         confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
```

```
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._get_connection() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._get_connection() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

```

```
237             if filters.get("event_type"):
238                 # Check if there is an event of a specific type inside this segment
239                 query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
240                 params.append(filters["event_type"])
241
242             segments_list = []
243             with self._get_connection() as conn:
244                 cursor = conn.cursor()
245                 rows = cursor.execute(query, params).fetchall()
246
247                 for row in rows:
248                     r_dict = dict(row)
249                     r_id = r_dict["id"]
250                     r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
251
252                     # Fetch associated events
253                     event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
254                     r_dict["events"] = []
255                     for er in event_rows:
256                         e_dict = dict(er)
257                         e_dict["details"] = json.loads(e_dict["details"] or "{}")
258                         r_dict["events"].append(e_dict)
259
260                     segments_list.append(r_dict)
261
262             return segments_list
263
264         def clear_video_data(self, video_id: str):
265             with self._get_connection() as conn:
266                 conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
267                 conn.commit()
268
```

3. user (2026-06-10T06:10:55.464Z)

```
run_phase3_eval.py:3:Walks the `manifest.json` produced by sports-data-collector's `curate
export`,
docs\ROADMAP.md:53:`run_phase3_eval.py` walks the manifest, processes each video, and scores
detected
docs\CODE_MAP.md:89:- `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score,
aggregate. `load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as
label); config via `backend.config.load_config`; `segmenter_cls(db, cfg)` receives the typed
`AppConfig` (same object the live pipeline feeds segmenters).
docs\CODE_MAP.md:108:[Omitted long matching line]
docs\CODE_MAP.md:173:[Omitted long matching line]
docs\CODE_MAP.md:242:- **Reporting never breaks a run:** `emit_indexer_report` swallows all
exceptions (designed for `main.py` `finally:`) and is emitted even on validation failure.
`obsreport` is a SOFT dep ? absent -> no-op.
docs\PLATFORM_ARCHITECTURE.md:155:  their own VM/GPU* is a meaningful ongoing load (updates,
monitoring, firewall/NAT, the
docs\PLATFORM_ARCHITECTURE.md:506:  pool by default**, keep the training corpus a **separate,
firewalled, opt-in sub-pool**,
backend\eval\batch.py:3:Phase 3 of the collector->indexer bridge: walk the `manifest.json`
emitted by
docs\GOLDEN_SET_VENDORING.md:44:- **Edge cases:** lets/replays, service faults, between-point
walking, coaching pauses, mid-rally net-cord, broadcast replays and graphics.
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:201:gym reverb + background
games, so audio loudness ? *court occupancy*, not *rally*. Same wall as E1 (~2.2× ceiling).
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md:229:- **Calibration, not
representation, is the n=2 wall** ? learned probabilities separate (AUC 0.84) but the operating
```

4. user (2026-06-10T06:11:14.030Z)

```
run_process_and_stitch.py:69:        db.add_video(video_id, video_path, "processed", [])
run_phase3_eval.py:143:                db.add_video(video_id, video_path, "processed", [])
tests\test_automated_provider.py:84:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_automated_provider.py:85:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_automated_provider.py:86:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_database.py:21:    vid_added = db.add_video("vid1", "vid1.mp4", "pending", [{"val":
1}])
tests\test_database.py:35:    db.add_video("vid1", "vid1.mp4", "pending", [])
tests\test_database.py:37:    sid = db.add_play_segment("vid1", 10.0, 20.0, "PlayerA",
"PlayerB", {"foo": "bar"})
tests\test_database.py:40:    db.add_event(sid, 15.0, "smash", "PlayerA", {"speed": 300})
tests\test_database.py:114:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:116:    cid = db.add_candidate_segment(
tests\test_database.py:141:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:143:    cid = db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
tests\test_database.py:144:    sid = db.add_play_segment("vid1", 1.0, 4.0)
tests\test_database.py:160:    db.add_video("vid1", "vid1.mp4", "processed", [])
tests\test_database.py:161:    db.add_candidate_segment("vid1", "yolo_hybrid", 1.0, 4.0)
tests\test_eval_harness.py:17:    d.add_video("vid1", "/x/match.mp4", "processed", [])
tests\test_eval_harness.py:27:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:28:    db.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_harness.py:33:    db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
tests\test_eval_harness.py:34:    db.add_candidate_segment("vid1", "tracknet_hybrid", 40.0,
50.0)
tests\test_eval_harness.py:48:    db.add_candidate_segment("vid1", "yolo_hybrid", 0.0, 10.0)
tests\test_eval_harness.py:49:    db.add_candidate_segment("vid1", "yolo_hybrid", 20.0, 30.0)
tests\test_eval_harness.py:50:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:65:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:71:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:87:    db.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_harness.py:121:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_harness.py:122:    d.add_play_segment("vid1", 0.0, 10.0)
backend\main.py:112:            db_instance.add_video(video_id, req.video_path, "failed",
[r.dict() for r in val_results])
backend\main.py:123:        db_instance.add_video(video_id, req.video_path, "processed",
[r.dict() for r in val_results])
backend\main.py:369:        db_instance.add_video(video_id, args.video_path, status, [r.dict()
for r in val_results])
tests\test_eval_regression.py:14:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:16:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:17:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:56:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:57:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:69:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:70:    d.add_play_segment("vid1", 0.0, 10.0)
tests\test_eval_regression.py:71:    d.add_play_segment("vid1", 20.0, 30.0)
tests\test_eval_regression.py:111:    d.add_video("vid1", "/x/m.mp4", "processed", [])
tests\test_eval_regression.py:112:    d.add_play_segment("vid1", 0.0, 10.0)
backend\infrastructure\database.py:103:    def add_video(self, video_id: str, filepath: str,
status: str, validation_results: List[Dict[str, Any]]) -> bool:
backend\infrastructure\database.py:116:    def add_play_segment(self, video_id: str, start_time:
float, end_time: float, server: Optional[str] = None, receiver: Optional[str] = None, metadata:
Optional[Dict[str, Any]] = None) -> Optional[int]:
backend\infrastructure\database.py:130:    def add_event(self, segment_id: int, timestamp:
float, event_type: str, player: Optional[str] = None, details: Optional[Dict[str, Any]] = None)
-> bool:
backend\infrastructure\database.py:143:    def add_candidate_segment(self, video_id: str,
detector: str, start_time: float, end_time: float,
tests\test_gemini.py:19:    db_mock.add_play_segment.return_value = "segment_1"
tests\test_gemini.py:58:    db_mock.add_play_segment.side_effect = ["segment_1", "segment_2"]
backend\pipeline\segmenters\yolo_hybrid.py:466:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\yolo_hybrid.py:551:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\wasb_hybrid.py:145:                candidate_ids[i] =
```

```
self.db.add_candidate_segment(
backend\pipeline\segmenters\wasb_hybrid.py:212:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
tests\test_yolo_hybrid.py:63:    db_mock.add_play_segment.return_value = "yolo_segment_1"
tests\test_yolo_hybrid.py:131:    db_mock.add_play_segment.return_value = "segment_ok"
tests\test_yolo_hybrid.py:199:    db_mock.add_candidate_segment.return_value = 77
tests\test_yolo_hybrid.py:200:    db_mock.add_play_segment.return_value = 101
tests\test_yolo_hybrid.py:222:    assert db_mock.add_candidate_segment.called
tests\test_yolo_hybrid.py:223:    _, kwargs = db_mock.add_candidate_segment.call_args
tests\test_yolo_hybrid.py:250:    db_mock.add_play_segment.return_value = 101
tests\test_yolo_hybrid.py:271:    db_mock.add_candidate_segment.assert_not_called()
backend\pipeline\segmenters\motion.py:175:            segment_id =
self.db.add_play_segment(video_id, start, end, server, receiver, metadata)
backend\pipeline\segmenters\motion.py:181:                self.db.add_event(
backend\pipeline\segmenters\motion.py:194:                    self.db.add_event(
backend\pipeline\segmenters\motion.py:207:                self.db.add_event(
backend\pipeline\segmenters\tracknet_hybrid.py:164:                candidate_ids[i] =
self.db.add_candidate_segment(
backend\pipeline\segmenters\tracknet_hybrid.py:238:                segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
backend\pipeline\segmenters\gemini.py:418:            segment_id =
self.db.add_play_segment(video_id, start, end, None, None, metadata)
tests\test_tracknet_hybrid.py:60:    db.add_candidate_segment.return_value = 55
tests\test_tracknet_hybrid.py:61:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:69:    _, kwargs = db.add_candidate_segment.call_args
tests\test_tracknet_hybrid.py:127:    db.add_play_segment.return_value = 200
tests\test_tracknet_hybrid.py:132:    db.add_candidate_segment.assert_not_called()
tests\test_tracknet_hybrid.py:150:    db.add_candidate_segment.return_value = 55
tests\test_tracknet_hybrid.py:151:    db.add_play_segment.return_value = 200
```

5. user (2026-06-10T06:11:15.273Z)

```
108:- **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
```

6. user (2026-06-10T06:11:22.442Z)

```
80      - Small corpus ? tiny held-out set ? high metric variance; the guardrail is correct
81        but the numbers will be noisy until the golden set grows.
82      - MD5-only dedup is insufficient (re-encodes evade it) ? must track match identity.
83
84      ### Verification
85      - Unit test: overlapping train/eval ids ? `assert_disjoint` raises.
86      - Integration: a deliberately-leaked manifest fails the training entrypoint.
87
88      ---
89
90      ## 3. Async job model (debt #16)
91
92      ### Problem
93      `POST /api/process` runs validate ? segment ? report **inline**, blocking the
94      request for the full duration of a long video. Unworkable as a hosted/product API.
95
96      ### Approach (minimal first slice ? single self-hosted box)
97      - `POST /api/process` returns **202 + `job_id`**; add `GET /api/jobs/{job_id}`
98        returning `{status, progress, result, reports}`.
99      - Run the pipeline on a background worker ? start with a `ThreadPoolExecutor`
100       (segmenters already use threads internally for AI handoff, so this fits); leave a
101       real queue (RQ/Celery) for later if multi-process is needed.
102     - Persist job state in the existing SQLite DB via the migration ladder
103       (`backend/infrastructure/database.py::_init_db`, bump `PRAGMA user_version` 1 ? 2,
104       add a `jobs` table). The observability report becomes the job's artifact.
105     - Update the static UI to poll `GET /api/jobs/{id}` instead of blocking.
106
107     ### Risks
108     - **SQLite thread-safety:** verify `check_same_thread`/connection-per-thread;
```

```
109       `_get_connection` already re-issues `PRAGMA foreign_keys=ON` per connection ?
110       confirm it's safe under the executor.
111     - Cancellation/timeout and crash recovery (a job in RUNNING after a restart).
112     - In-process store is not horizontally scalable ? acceptable for the current
113       single-box self-hosted target; call it out so it isn't mistaken for production-HA.
114
115     ### Verification
116     - Submit a job ? poll ? DONE with the same segments the sync path produced.
117     - Server restart mid-job leaves a recoverable/clearly-FAILED job, not a hang.
118
119     ---
120
121     ## Out of this doc
122     - **Detector/WSL abstraction (#6):** in flight (PR #49).
123     - **Unify hybrid segmenters (#7 in LOW_REGRET):** owner-deferred.
124     - **ByteTrack `supervision<0.30.0` migration + real-footage validation (#14/#15):**
125       need a GPU/WSL box and real video ? owner-machine work, tracked separately.
126
```

7. user (2026-06-10T06:11:23.366Z)

```
540                     if shot_count is None: shot_count = max(3, int(dur / 0.8))
541                     else:
542                         try: shot_count = int(shot_count)
543                         except (ValueError, TypeError): shot_count = max(3, int(dur / 0.8))
544
545                     metadata = {
546                         "shot_count": shot_count,
547                         "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
548                         "detector": f"yolo-hybrid-{model_name}"
549                     }
550
551                     segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
552                     if segment_id:
553                         # Link back to the candidate that produced this segment. A candidate
554                         # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
555                         # the last one ? the link table records "this candidate was
confirmed".
556                         candidate_id = segment.get("_candidate_id")
557                         if candidate_id is not None:
558                             self.db.link_candidate_to_segment(candidate_id, segment_id)
559                         segments_data.append({
560                             "id": segment_id,
561                             "start_time": start,
562                             "end_time": end,
563                             "duration": dur,
564                             "metadata": metadata
565                         })
566                 except Exception as e:
567                     logger.warning(f"Skipping invalid segment struct: {e}")
568
569             return segments_data
```

8. user (2026-06-10T06:11:43.083Z)

```
3:**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted
video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.
28:  -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
94:  - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`,`indexing`,`output`,`stitching`
```

(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
108:[Omitted long matching line]
113:- `@backend/pipeline/segmenters/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI,
no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded;
`max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates; `parse_time` accepts
colon timestamps (warns); does NOT persist candidate segments (only hybrids feed the fine-tuning
table).
121:[Omitted long matching line]
130:- `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)`,
`::default_grid` (45 cfgs), `::make_predict_fn` (parses trajectory once, closes over
`trajectory_to_action_windows`), `::load_golden_gts` (§`start_time,end_time` cols; ? SILENTLY
skips bad rows ? opposite of annotations.load_annotations). ? `{load_golden_gts,
make_predict_fn, BASELINE}` imported by 4 downstream drivers (calibrate_local,
export_wasb_segments, gemini_refine, audio/e1_probe).
150:- `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure, unit-
tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::ClipWindow`(rally:str), `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (?
duplicate compiler's by copy). `_read_time -> backend/eval/annotations.parse_timestamp`.
166:[Omitted long matching line]
173:[Omitted long matching line]
178:[Omitted long matching line]
202:- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`,
`export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`;
SILENTLY skips bad rows.
239:- **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
242:- **Reporting never breaks a run:** `emit_indexer_report` swallows all exceptions (designed
for `main.py` `finally:`) and is emitted even on validation failure. `obsreport` is a SOFT dep ?
absent -> no-op.
243:- **spec.yaml <-> harvest.py coupling:** rule `path` strings dotted-navigate harvest's
stage/metric/detail names; rename a detail in harvest.py and the matching rule silently stops
firing. Add rules in spec.yaml, not code.
244:- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks
(30.0, 1280.0) silently mis-scale velocity thresholds; calibrate_wasb CLI bakes `59.94/1920`.
The report's `fps_fallback_used` flag surfaces this.
286:| Add a DB column/table | `@backend/infrastructure/database.py::Database._init_db` migration
ladder + bump `PRAGMA user_version` (else version-assert tests fail) |
320:| `test_database.py` | `infrastructure.database.Database`
schema/CRUD/migrations/`user_version`/`EXPECTED_CANDIDATE_COLUMNS` guard |

9. user (2026-06-10T06:11:44.313Z)

6:import uvicorn
91:@app.post("/api/process")
92:def process_video(req: ProcessRequest):
172:@app.post("/api/query")
183:@app.post("/api/compile")
423:        uvicorn.run(app, host="127.0.0.1", port=args.port)

10. user (2026-06-10T06:12:30.138Z)

Structured output provided successfully

11. assistant (2026-06-10T06:12:33.777Z)

Verification complete ? structured output submitted.